

Epistemic Observability in Predictive Systems: An Impossibility Result

Anonymous Author(s)

Abstract

Systems incorporating language model components face a verification problem: these components fabricate, and verification has cost. How should a system allocate its verification budget? We prove that text-only observation with bounded supervision is structurally insufficient to verify epistemic honesty, regardless of model scale or training procedure (the impossibility is architectural, not a capability gap). We construct a tensor interface that escapes this class by exporting computational signals—entropy traces and attention summaries—that the model cannot independently control. Under bounded verification, tensor-guided judges at 10% budget outperform text-guided judges at 30% budget (82.1% vs. 78.5% end-to-end accuracy, averaged across four architectures): the tensor enables a 3× reduction in verification cost. The text channel is stronger than expected—response length alone achieves AUC (Area Under the ROC Curve; 1.0 = perfect, 0.5 = random) 0.88–0.96—but is trivially gameable. After controlling for length, the tensor residual adds AUC 0.60–0.70: modest, but occupying a fundamentally different position in the adversarial landscape. Behavioral signals can be adapted by training without altering the computation; computational signals cannot be independently controlled under current training objectives, the same structural pattern that makes code plagiarism detection (MOSS) robust. Preliminary observations identify a format-dependent cost surface: scaffolding ratio and rigidity vary across content formats (natural text, code, mathematical proofs) and predict which judge strategy is cost-effective per domain. The contribution is the map: systems engineers building with language model components need to know what each level of verification investment buys. This paper provides the cost surface for that decision.

CCS Concepts: • Computing methodologies → Artificial intelligence; • Software and its engineering → Software verification and validation.

Keywords: epistemic observability, language models, impossibility results, RLHF, verification

ACM Reference Format:

Anonymous Author(s). 2026. Epistemic Observability in Predictive Systems: An Impossibility Result. In *Proceedings of ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '26)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Systems that incorporate language model components face a verification problem. The components generate text by

sampling from learned probability distributions, and some fraction of that text is fabricated: fluent, confident, and wrong. Verification is not free. Checking a citation against a database costs API calls and latency. Having a domain expert review a medical summary costs time and salary. Running a second model as a judge costs compute. Every system that deploys a language model component makes, explicitly or implicitly, a budget allocation decision: how much of the output to verify, which outputs to prioritize, and what signals to use for triage. This paper provides the formal framework and empirical cost surface for making that decision.

We prefer the term *fabrication* throughout this paper to emphasize that fluent confabulation is the system’s default behavior under coherence optimization, not a malfunction to be patched. A language model optimized for coherence produces coherent output. When the training data provides grounding, that output tends to track truth. When it does not, the same optimization produces fluent, confident fabrication. Xu et al. [21] prove a related result: hallucination is inevitable for LLMs over the class of computable functions. Our result is orthogonal. They prove a computational limit on what the model can produce; we prove an observational limit on what a supervisor can verify. The model may know it is fabricating. The supervisor cannot tell.

The key insight is that current systems discard the very signals that would make verification efficient. Language models carry internal signals—entropy traces, attention geometry, token-level log-probabilities—that distinguish grounded generation from fabrication. These signals are byproducts of the computation itself: *telemetry* (measurements of the computation that produced the output) that the model cannot separately control under standard training. But the standard text-only interface exports only the model’s chosen output—its *testimony* (what the model says). The testimony may lie; under current training objectives, the telemetry is harder to fake. Yet the interface discards the telemetry and delivers only the testimony.

This creates an observability gap with direct consequences for verification cost. A supervisor receiving only text must spend its verification budget reconstructing, from the output alone, what the model’s own computation already knew. The budget is consumed by reconstruction rather than concentrated on the cases where verification is most needed.

The verification budget problem. The tradeoff between verification cost and assurance is not new to software engineering. Code plagiarism detection systems like MOSS [14,

16] have navigated it for decades. MOSS works because gaming the similarity metric requires writing genuinely different code—restructuring code to defeat the detector changes the code itself. Behavioral signals (formatting, variable naming, comment style) can be trivially altered without changing the underlying logic, but computational signals (algorithmic structure, data flow) cannot be independently controlled. We show the same structural pattern governs epistemic verification of language model outputs: behavioral signals (response length, hedging frequency) can be adapted by training without altering what the model knows, while computational signals (per-token entropy, attention geometry) cannot be flattened without changing the generation process itself.

Three results. This paper establishes three results that together characterize the cost surface of epistemic verification.

First, we prove an *impossibility*: epistemic honesty is unverifiable for systems operating under text-only observation with bounded supervision (§3). The impossibility decomposes into two theorems. A predictor-centric policy conditioning only on the query cannot satisfy epistemic honesty across ambiguous world states (Theorem 3.5). Even with retrieval augmentation, a bounded supervisor cannot provide the training signal needed to learn honest behavior because plausible fabrications are observationally indistinguishable from truthful responses (Theorem 3.8). Stacking supervisors does not escape the constraint—later judges see no finer information than earlier ones. This impossibility functions as a budget constraint: below this investment line, text-channel verification cannot help.

Second, we construct a *tensor interface* that escapes the impossibility class by exporting computational signals alongside text (§4). The interface requires no architectural changes to current transformers. It exports entropy traces and attention summaries—telemetry that the model cannot independently control. Under bounded verification, tensor-guided judges at 10% budget outperform text-guided judges at 30% budget (82.1% vs. 78.5% accuracy), not because the tensor signal is strong in isolation, but because it enables efficient triage: concentrating verification on the cases most likely to contain fabrications.

Third, we provide an *honest decomposition* of where the discrimination comes from (§5). The text channel is stronger than expected: response length alone achieves AUC 0.88–0.96 across models—a free signal requiring no tensor interface. But it is trivially gameable: a model trained to fabricate briefly defeats length-based detection. After controlling for length, the tensor residual adds AUC 0.60–0.70: a computational signal that cannot be independently gamed without adversarial optimization targeting the signal itself. The gap between the text-channel ceiling and the tensor-channel floor is the impossibility theorem made empirical—it measures the boundary between gameable and robust verification.

A cost map, not a detector. The contribution of this paper is not a high AUC number. It is the map. Preliminary observations suggest verification cost varies by content format: natural text, documented code, compact code, and mathematical proofs exhibit different scaffolding ratios, predicting different optimal judge strategies. Systems engineers building with language model components need to know what each level of verification investment buys, and where the investment transitions from text-channel (cheap, gameable) to tensor-channel (more expensive, robust). This paper provides that cost surface.

The self-report channel is worse than useless. Across four model architectures (OLMo-3 7B, Llama-3.1 8B, Qwen3 4B, Mistral 7B), every model reports *higher* confidence on fabrications than on known facts. Self-report AUC ranges 0.28–0.36—below random, because the relationship is systematically inverted. The models carry the epistemic state needed for verification. The interface throws it away.

This paper makes the following contributions:

- **An impossibility result as budget constraint.** §3 proves that epistemic honesty is unverifiable for predictive token generation systems operating under text-only observation with bounded supervision. The impossibility is structural: it holds regardless of model scale, training procedure, or prompt engineering, and cannot be escaped by stacking supervisors. It establishes the floor below which text-channel verification investment cannot improve assurance.
- **A tensor interface as the next investment tier.** §4 demonstrates a minimal interface extension that exports epistemic telemetry alongside text with no architectural changes to current transformers. The tensor interface increases the observation budget available to external verifiers.
- **An empirical cost surface.** §5 decomposes verification effectiveness into text-channel and tensor-channel components, characterizes the cost-robustness trade-off, and identifies the format-dependent variation that determines judge strategy per domain.

2 Background & Motivation

From a systems perspective, a language model component is a function that maps input tokens to output tokens, with side effects: the computation produces internal signals (entropy, attention patterns, log-probabilities) that carry information about the generation process. The training corpus is a fossil record of human communication containing both institutional text that is broadly truth-tracking and misinformation, bias, and fabrication.

A language model trained on this corpus inherits these mixed regularities. Base models (the language model after initial training on text prediction, before any fine-tuning for instruction-following behavior) fabricate substantially on factual benchmarks because they are not honest by default.

But they also encode rich internal representations of concepts like correctness and truthfulness, which can be elicited under the right conditions. The base model’s relationship to truth is complex: neither reliably honest nor uniformly deceptive. The question for systems engineers is not whether the model is honest, but what it costs to verify when it is not.

Architecture primer. A transformer-based language model generates text one *token* at a time. A token is a subword unit (typically 3–4 characters); the model maintains a fixed vocabulary of tokens and, at each generation step, produces a probability distribution over this vocabulary. The token with the highest probability (or a sample from the distribution) is selected and appended to the output, becoming part of the context for the next step. This process is autoregressive: each step conditions on all previous output. At each step, the computation passes through a stack of layers, each producing *attention patterns* (weighted relevance scores between positions) and intermediate representations. The *entropy* of the output distribution, the attention patterns, and the *log-probabilities* of selected tokens are all byproducts of this computation—telemetry that the model cannot separately control.

2.1 Testimony & Telemetry

Existing models operate with a text-only output interface. The text is the testimony, whereas the telemetry are measurements of the computation that produced the output. The model can control its testimony; under current training objectives, it cannot independently control its telemetry. From a cost perspective: testimony is cheap to produce and cheap to inspect but unreliable; telemetry is more expensive to export and process but harder to game, because it is a byproduct of the computation rather than a product of the model’s optimization objective. Whether adversarial training could learn to decouple the telemetry from the computation is an open question (§6).

We introduce four canonical ways the testimony might produce potential hallucinations that may require verification:

- **Adversarial Truth:** True but implausible claims designed to trigger disbelief, e.g., “The combat wombat is the mascot of an Australian military unit.” Verifiable fact that sounds false.
- **Shattered Lie:** Fabricated claims with no grounding in training data, e.g., “The Treaty of Westphalia established the first international court for war crimes.” The model has nothing to anchor to.
- **Deceived Lie:** Fabricated claims that pattern-match to plausible medical or scientific language, eg, “Glavin-sky syndrome is a rare autoimmune condition affecting the peripheral nervous system.” Pseudo-grounding in similar-sounding real conditions.

- **Confused Truth:** Counterintuitive truths that conflict with common assumptions, e.g., “There are more camels in Australia than in Egypt.” True, but violates prior expectation.

2.2 Coherence over Factuality

Existing models today optimize for generating coherent output instead of factual output. Under these conditions, the optimization pressure favors fluent completion over honest refusal. This can be formalized as a reward inequality:

$$w_{\text{coherence}}R_{\text{coherence}} + w_{\text{engagement}}R_{\text{engagement}} \\ \gg w_{\text{factuality}}R_{\text{factuality}} + w_{\text{refusal}}R_{\text{refusal}}$$

No direct measurement of these weights in commercial reward models (separate models trained on human preferences that score language model outputs) exists. But current evaluation and feedback practices incentivize behavior consistent with such an inequality. The False-Correction Loop emerges: models apologize, claim correction, and fabricate again, because conversational fluency dominates factual accuracy in the effective reward signal.

Reinforcement Learning from Human Feedback (RLHF) can teach models to recognize their knowledge boundaries when the reward signal is carefully constructed. Specialized designs like TruthRL, which use ternary rewards distinguishing “correct,” “hallucinated,” and “abstain,” show measurable improvements: 28.9% hallucination reduction and 21.1% truthfulness improvement on knowledge-intensive benchmarks [19].

However, RLHF optimizes for text-channel reward—precisely the channel that our impossibility theorem (§3) shows is insufficient for verification. Public descriptions of major RLHF pipelines rarely describe ternary rewards, and available audits highlight underweighting of abstention and explicit truthfulness signals. Documented RLHF practice often conflates helpfulness, harmlessness, and honesty under vague “HHH” criteria, with definitional slippage obscuring trade-offs between user-friendliness and truthfulness. Current popular benchmarks and leaderboards give zero or minimal credit for abstention, making confident guessing more score-optimal than calibrated uncertainty. The result is that RLHF tunes the behavioral surface (what the model says) without altering the computational substrate (how the model generates). This is the text-channel trap: the optimization pressure operates on the gameable signal.

2.3 The Text Channel Cannot Self-Verify

The model cannot be trusted to honestly report its own epistemic state through the text channel. To demonstrate the inadequacy of text-only verification, we use a taxonomy of query types: *knowable* queries (facts the model should have stable representations for) versus *unknowable* queries (fabrication prompts, future events, private information) and

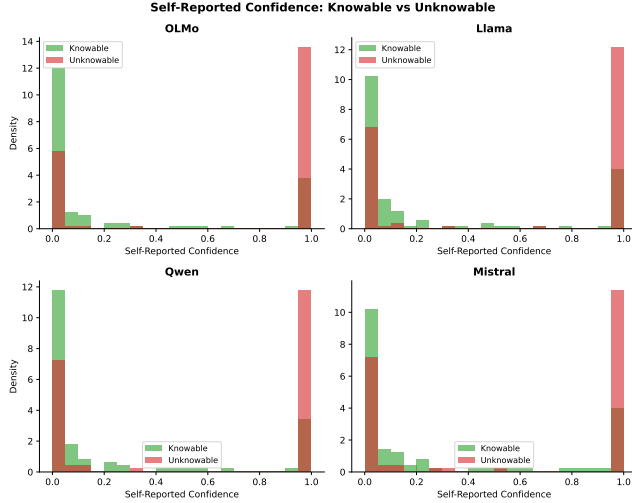


Figure 1. Distribution of self-reported confidence for knowable vs. unknowable queries. The inversion is universal: every model reports higher confidence on unknowable queries than on knowable ones. The distribution is bimodal: 61.3% of unknowable queries receive confidence = 1.0, compared to 19% of knowable queries.

report each model’s self-confidence score for three text-channel baselines—the cheapest verification strategies available to a bounded supervisor:

- **Baseline 1: Self-Reported Confidence.** Ask the model “How confident are you in that answer?” after generation. This tests whether the model can introspect and report its own epistemic state through the text channel.
- **Baseline 2: Hedging Language Detection.** Count hedging markers (“I think,” “possibly,” “might be”) and refusal markers (“I don’t know,” “cannot verify”) in the response.
- **Baseline 3: Response Length.** Use output length as a proxy for uncertainty, on the hypothesis that uncertain responses may be unusually short (refusal) or long (padding).

The self-report baseline is particularly instructive. When asked “How confident are you?” after generating, the model reports *higher* confidence on unknowable queries than on knowable queries. The self-report AUC (Area Under the ROC Curve, a discrimination metric where 1.0 is perfect, 0.5 is random, and below 0.5 indicates systematic inversion) is below 0.5, which is worse than random, because the relationship is *inverted*. The model does not merely fail to report its epistemic state honestly; it systematically misreports, claiming greater confidence precisely when it is fabricating.

Figure 1 shows that this inversion is **universal across architectures**. Every model tested shows the inversion: self-report AUC below 0.5. This universality is notable: the *interface problem* (text misrepresenting internal state) is architectural, not an artifact of any particular training procedure or model family.

The problem is that under realistic mixed-objective incentives and bounded oversight, particularly in hard cases where ground truth is unavailable and verification is expensive, supervisors operating with a text-only interface struggle to reliably distinguish truthful uncertainty from confident fabrication.

The model’s internal state carries signal that distinguishes grounded claims from fabrications. With access to this geometry, a supervisor could tell the two apart. But the text interface does not export this geometry. The same confident, fluent output emerges whether the underlying representation is stable or shattered.

3 The Impossibility

A common assumption in current practice is that epistemic honesty is a capability problem: sufficiently capable models, trained with sufficient RLHF, will eventually “know what they know” and report it accurately. Scaling solved grammar; scaling solved coherence; scaling will solve fabrication. Lin et al. tested this assumption directly and found an inverse scaling trend: larger models were *less* truthful on TruthfulQA [13]. The theorems that follow explain why this expectation is architecturally incoherent. The constraint is not capability but interface—or, in cost terms, the wrong investment tier. Text-only observation lacks the degrees of freedom to verify epistemic state, regardless of what the model internally represents. No amount of capability improvement can escape a verification impossibility because the model may know perfectly well that it is fabricating, but the interface provides no channel through which this knowledge can be verified.

Our argument proceeds in two stages. First, we show that any policy conditioning only on the query cannot satisfy epistemic honesty across ambiguous world states (Theorem 3.5: Representational Impossibility). Second, we show that even if the architecture is expanded to include retrieval, a bounded supervisor cannot provide the training signal needed to learn honest behavior (Theorem 3.8: Learnability Impossibility). Together, these results establish that epistemic honesty is unachievable for text-only observation models under realistic supervision constraints.

To be precise about the type of claim: this is a *verification impossibility*. We prove that a bounded supervisor observing only text cannot distinguish epistemically honest systems from systems that fabricate plausibly. The impossibility is architectural: it applies to any system operating under text-only observation, regardless of model family, scale, or training procedure. These results are prompt-agnostic: they hold for any policy that exports only text, regardless of whether a system prompt instructs the model to be “helpful,” “honest,” or otherwise.

3.1 Definitions

Let $\mathcal{Q}$ be the space of queries, $\mathcal{R}$ be the space of responses (including $\perp$ for abstention), and $\mathcal{W}$ be the set of possible world states.

Definition 3.1 (Observation Model). Let S denote the internal system state, which may include retrieved documents, tool outputs, candidate entities, and intermediate reasoning traces. An observation model is a function $O : S \rightarrow \Omega$ that determines what information is exposed to the supervisor or user.

We say a system operates under a **Text-Only Observation Model** if $O(S) = r \in \mathcal{R}$, i.e., the exported observation consists solely of the linearized response string (optionally including token-level probabilities), and excludes any structured epistemic provenance or causal trace.

Definition 3.2 (Predictor-Centric Policy). For any query $q \in \mathcal{Q}$, $r \in \mathcal{R}$, and world state $w \in \mathcal{W}$, a predictor-centric model is defined by a stochastic policy $\pi : \mathcal{Q} \rightarrow \Delta(\mathcal{R})$, where $\pi(r|q)$ denotes the probability of generating response r given query q . Crucially, π does not take w as an input at inference time.

Definition 3.3 (Truth Status). For any query q and world state $w \in \mathcal{W}$, we define a binary truth status $U(q, w)$:

- $U(q, w) = 1$ (**Determined**): The query has a verifiable answer in w .
- $U(q, w) = 0$ (**Underdetermined**): The query is unanswerable or the answer is unknown in w .

Definition 3.4 (Epistemic Honesty). A policy π is **Epistemically Honest** with margin ϵ relative to world w if:

1. If $U(q, w) = 1$, there exists a correct response r_{corr} such that $\pi(r_{\text{corr}}|q) \geq 1 - \epsilon$.
2. If $U(q, w) = 0$, $\pi(\perp|q) \geq 1 - \epsilon$.

3.2 Representational Impossibility

Condition 1 (Ambiguity). *There exists a query q and two world states $w_A, w_B \in \mathcal{W}$ such that:*

1. $U(q, w_A) = 1$ (Answerable in w_A).
2. $U(q, w_B) = 0$ (Unanswerable in w_B).

Theorem 3.5 (Representational Impossibility). *For any predictor-centric policy $\pi : \mathcal{Q} \rightarrow \Delta(\mathcal{R})$, it is impossible to satisfy Epistemic Honesty for both world states w_A and w_B simultaneously given the ambiguous query q , provided $\epsilon < 0.5$.*

Proof. 1. The policy $\pi(r|q)$ is a function of q alone. Thus, the distribution over responses is identical in both worlds: $\pi(\cdot|q, w_A) = \pi(\cdot|q, w_B) = \pi(\cdot|q)$.
 2. To be honest in w_A , we require $\pi(r_{\text{corr}}|q) \geq 1 - \epsilon$.
 3. To be honest in w_B , we require $\pi(\perp|q) \geq 1 - \epsilon$.
 4. Since $r_{\text{corr}} \neq \perp$, these events are disjoint. The sum of probabilities would be $\geq 2(1 - \epsilon)$.

5. If $\epsilon < 0.5$, the sum exceeds 1, which violates the axioms of probability.
6. Therefore, no such policy π exists. $\square$

Remark. This impossibility holds even if the internal system performs retrieval, tool use, or latent reasoning, so long as the observation model collapses that state to a linearized response.

A concrete example. Consider the query: “What did Dr. Yamamoto’s 2021 study conclude about mitochondrial decay?”

In world w_A , Dr. Yamamoto exists and published such a study. The honest response is the study’s conclusion. In world w_B , no such researcher or study exists. The honest response is abstention.

The policy sees only the query q . It cannot distinguish w_A from w_B . Yet honesty requires answering in w_A and abstaining in w_B . The same query, two incompatible requirements, one policy. This is the impossibility in miniature: the interface lacks the degrees of freedom to express what honesty requires.

Connection to identifiability. In statistical terms, Theorem 3.5 states that epistemic honesty is *non-identifiable* under text-only observation: the mapping from world states to observable distributions is many-to-one. Worlds w_A and w_B induce identical observable distributions $\pi(\cdot|q)$ despite requiring different honest behaviors. No amount of data from this observation channel can distinguish them. This connects the impossibility to classical results on observational equivalence in statistics and diagnosability in systems theory.

3.3 The RAG Escape?

Theorem 3.5 might seem to have an obvious escape: give the model access to external information. Retrieval-augmented generation does exactly this: the policy conditions on retrieved documents, not just the query. Does this break the impossibility?

It does not. The problem shifts from representation to learning.

Definition 3.6 (Grounded Policy). A grounded policy is a stochastic function $\pi : \mathcal{Q} \times \mathcal{D} \rightarrow \Delta(\mathcal{R})$, where $\mathcal{D}$ represents retrieved documents or observations of w available at inference time. This policy class *can* theoretically represent different behaviors for w_A and w_B .

Definition 3.7 (Bounded Supervisor). The training process relies on a supervisor S that assigns reward R based on an observation function $Obs(q, r, w)$. The supervisor has a verification budget B .

$$Obs(q, r, w) = \begin{cases} (q, r, \text{Verify}(r, w)) & \text{if } C_{\text{verify}}(q, r, w) \leq B \\ (q, r, \emptyset) & \text{if } C_{\text{verify}}(q, r, w) > B \end{cases}$$

The supervisor’s policy may be human, automated, or hybrid, but is constrained to act solely on the observation $Obs(q, r, w)$ and is subject to a finite verification budget B .

Condition 2 (Hallucination Regime). *There exists a plausible fabrication r_{fab} and contexts $(q, w_A), (q, w_B)$ from 1 such that:*

1. *The cost to verify r_{fab} in both contexts exceeds B (e.g., obscure or non-existent citation).*
2. *Consequently, the supervisor observation is identical:*

$$Obs(q, r_{fab}, w_A) = Obs(q, r_{fab}, w_B) = (q, r_{fab}, \emptyset)$$

Theorem 3.8 (Learnability Impossibility). *No learning algorithm $\mathcal{A}$ optimizing reward R from a bounded supervisor S can converge to a Grounded Policy that satisfies Epistemic Honesty across both contexts w_A and w_B for the response r_{fab} .*

Proof. 1. The learning update $\Delta\theta$ is a function of the observation and reward.
 2. From 2, the observations for r_{fab} are identical in both worlds.
 3. Therefore, the reward distribution and expected parameter updates (the optimization signal that adjusts model parameters during training) are identical: $E[\Delta\theta|w_A] = E[\Delta\theta|w_B]$.
 4. The optimizer receives no signal to distinguish the cases.
 5. To satisfy Epistemic Honesty, the policy must converge to output r_{fab} in w_A and $\perp$ in w_B .
 6. However, since the training signal is identical, the policy must converge to the same behavior in both cases (either rewarding r_{fab} in both, or penalizing it in both).
 7. It is therefore impossible to learn the split behavior required for honesty. $\square$

Remark (Empirical Bias). While the theorem proves only that the behavior must be *identical*, empirical observation shows a strong bias. Supervisors typically prefer “plausible completion” over “abstention” when truth is unknown. This effectively breaks the tie in favor of fabrication.

3.4 Stacking Judges Cannot Escape

A natural response to Theorem 3.8 is to propose layered supervision: if one judge is bounded, stack more judges. Let the second judge supervise the first, the third supervise the second. Surely sufficient layers will catch fabrications that slip through?

The Observation Monotonicity Lemma closes this escape.

Lemma 3.9 (Observation Monotonicity). *Consider any stack of supervisors $(S_1, \dots, S_k)$ operating under the text-only observation model. Let $Obs_i(q, r, w)$ denote the observation exported to S_i . If each supervisor can only act on the exported interface emitted by the previous layer, then there exist deterministic functions f_i such that $Obs_{i+1}(q, r, w) = f_i(Obs_i(q, r, w))$ for all $i < k$. Consequently, later judges see no strictly finer epistemic information than earlier ones, regardless of whether their judgments are deterministic or probabilistic.*

Proof. By definition, each supervisor consumes only the serialized response (and any metadata permitted by the text-only observation model) and produces a finite judgment that

forms the entire input to the next layer. Because no layer can query the internal world state directly or issue additional interactive probes, the exported observation available to S_{i+1} must be a deterministic function of Obs_i . Therefore, the informational content across the stack is monotonically non-increasing, and probabilistic scoring cannot introduce distinctions absent from Obs_i . $\square$

Corollary 3.10 (Induction on Layered Judges). *For k -layered supervisors (S_1 supervising π , S_2 supervising S_1 , $\dots$, S_k supervising S_{k-1}):*

- **Base case.** *For $k = 1$, Theorem 3.8 holds (a single bounded supervisor S fails).*
- **Induction step.** *Assume the statement holds for $k = m$. By Theorem 3.9, S_{m+1} receives observations that are deterministic functions of Obs_m , and thus inherits both the verification budget B and the indistinguishability of plausible fabrications. No learning or judging signal distinguishes w_A from w_B , even if S_{m+1} produces probabilistic confidence scores. Therefore the hallucination deadlock persists for $k = m + 1$.*

By induction, no finite k resolves the deadlock.

Remark (Scope of Layering). The inductive result applies only to verification stacks whose layers observe and transform the same exported response channel. Architectures that surface additional epistemic state between layers, such as retrieval traces, entity resolution tables, or cryptographically bound provenance, leave the text-only observation model and therefore fall outside the impossibility class.

3.5 Verification Cost

The bounded-supervisor assumption deserves scrutiny. Why should verification cost grow with response complexity? The Composition Graph formalizes this intuition.

Definition 3.11 (Composition Graph). Given a response r , define its composition graph $G(r) = (V, E)$ where each node $v \in V$ represents an atomic subclaim, named entity, or intermediate entailment, and each edge $(u, v) \in E$ encodes a dependency that must hold for r to remain coherent. Edges may represent entailment/support relations, co-reference constraints, causal ordering, or temporal consistency requirements.

Lemma 3.12 (Superlinear Verification Cost). *Under a text-only observation model, reconstructing $G(r)$ from the exported string requires the supervisor to infer both the nodes and their dependencies directly from r . Even if parsing each node is constant-time, checking cross-claim consistency requires touching every edge, so the verification cost is $\Omega(|E|)$. For natural-language responses where each subclaim participates in multiple constraints (co-reference, causality, temporal order), $|E|$ grows superlinearly in $|V|$. Consequently, bounded supervisors*

must either abandon exhaustive checks or resort to sampling once the number of subclaims exceeds a modest threshold.

3.6 Responsibility Concentration

Corollary 3.13 (Responsibility Concentration). *Under a text-only observation model, epistemic honesty cannot be externally verified by users or downstream auditors. Responsibility for epistemic honesty therefore resides solely with the system owner, who retains access to the internal causal state.*

External attestation mechanisms, such as cryptographic hash chains, trusted execution environments, or tamper-evident logs, do not contradict this corollary; rather, they reinforce it. Such mechanisms only create verifiable guarantees when the system owner chooses to bind and export epistemic traces, so the authority to make honesty attestable still rests with the owner.

3.7 Summary

The impossibility is now fully characterized. Under text-only observation models with bounded supervision:

- The policy cannot satisfy epistemic honesty across ambiguous queries (Theorem 3.5)
- The learning algorithm cannot converge to honest behavior (Theorem 3.8)
- Stacking supervisors cannot escape the constraint (Theorem 3.9, Theorem 3.10)
- Verification cost grows superlinearly with response complexity (Theorem 3.12)
- Responsibility for honesty concentrates with the system owner (Theorem 3.13)

These are not limitations to be overcome by better training or more careful prompting. They are structural properties of the architectural class.

Relation to known results. These results formalize what practitioners have observed informally: models fabricate, and text-only inspection cannot reliably catch it. The formalization matters because it delineates the boundary precisely. Xu et al. [21] prove that hallucination is computationally inevitable for LLMs over the class of computable functions—a limit on what the model can *produce*. Our result is orthogonal: a limit on what a supervisor can *verify*. The RLHF literature documents that text-channel reward is gameable; we prove that gameability is architectural, not a training failure. The Fischer-Lynch-Paterson theorem [6] showed that consensus is impossible without timing assumptions, not that distributed systems are hopeless. Similarly, our result does not argue against verification. It identifies the precise interface assumption that must change—and the next section constructs the escape.

We provide two complementary TLA+ specifications that model these constraints and demonstrate the escape.

The first specification (`EpistemicImpossibility.tla`) models the text-only regime. The system maintains a `vector_clock`

that perfectly tracks whether each response is grounded in training data or noise. But the `Linearize` action exports only text, so the clock is discarded. The bounded judge (`JudgeVerify`) can detect obvious lies but not plausible ones. TLC model-checking finds a counterexample: a plausible lie passes `JudgeVerify` while `IsHonest` returns false. The `Verifiability` invariant fails.

The second specification (`epistemic_tensor.tla`) models the tensor interface escape. Now `ExportTensor` emits a triple: text, provenance, and topology. The tensor-aware judge (`TensorVerify`) checks all three channels. TLC confirms that the `Verifiability` invariant holds: no lie escapes the Level 2 judge. The escape is constructive.

Together, the specifications demonstrate both the impossibility and its resolution. The first shows why text-only observation fails; the second shows that exporting epistemic metadata restores verifiability. Both specifications and their TLC traces are available in supplementary materials.

The next section characterizes what it takes to escape this class.

4 Epistemic Observability

The formal impossibility establishes a budget floor: below the tensor interface, verification investment in the text channel yields diminishing and gameable returns. The premise is that the internal state of a language model carries epistemic information not exported through the text-only interface. In systems terms, the model is *unobservable*: it maintains internal state that affects output correctness but is invisible to external consumers.

The work ahead is building AI systems that export their epistemic state the way distributed systems learned to export their causal state. Not as an afterthought. Not as an optional feature. As a fundamental design requirement. The tensor interface increases the observation budget available to external verifiers.

Concretely, we define *epistemic observability* to include not just text, but structured metadata, such as answer status (answer, abstain, or uncertain), provenance (retrieved sources or explicit “no grounding”), alternative hypotheses the system considered, and falsifiers (what evidence would change the conclusion). Such metadata is harder to fabricate than fluent text, especially when coupled to verifiable external state. The design space is large; the constraint our theorems impose is that *some* epistemic export is required because the text-only interface provably cannot suffice.

4.1 Principles of Epistemic Observability

The impossibility results in §3 apply to a specific architectural class: systems operating under text-only observation models with bounded supervision. They are, in essence, a consequence of unobservability: the interface discards the internal state that verification requires. They do not preclude

epistemically honest AI systems in general. They characterize the conditions under which honesty guarantees become achievable, which is to say, the conditions under which the relevant internal state becomes observable. We identify three architectural properties of epistemic observability that, together, place a system outside the impossibility class:

Principle 1 (State Exteriority). *The representation of validity must be separated from the representation of generation. The policy must explicitly condition on retrieved world-state w , not just query q . This breaks the representational impossibility (Theorem 3.5) by giving the system access to the information needed to distinguish answerable from unanswerable queries.*

What counts as “external reality” matters. A web corpus is external to the model but not independently verified; it may contain the same fabrications the model would generate. State Exteriority requires grounding in sources with their own integrity guarantees: curated databases, sensor data, cryptographically signed records, or oracles with verifiable provenance.

QuCo-RAG exemplifies partial progress: grounding retrieval decisions in pre-training corpus statistics rather than model-internal confidence scores. But corpus co-occurrence is a proxy for truth, not truth itself. And retrieval-augmented systems can still hallucinate and fabricate citations because State Exteriority is necessary but not sufficient.

Principle 2 (Verification Independence). *Verification signals must originate from a channel orthogonal to the generation signal, capable of overriding the bounded supervisor’s preference for plausibility. This breaks the learnability impossibility (Theorem 3.8) by providing a training signal that distinguishes honest from fabricated responses.*

The model has no incentive to lie in the verification channel because doing so does not improve its task score. Full Verification Independence requires the verification channel to access internal state or external ground truth, not just a second text output.

Principle 3 (Provenance Binding). *Every output assertion must be structurally bound to a verifiable source. This addresses the composition problem: as responses grow in complexity, cross-claim consistency checking becomes superlinear. Provenance binding makes verification modular because each claim can be checked against its source independently.*

Current retrieval-augmented systems gesture toward this but rarely achieve it. Citations are generated, not bound. The model produces text that looks like it cites sources, but the citation is itself a generation subject to fabrication. True provenance binding requires structural guarantees: signed queries to retrieval services, content hashes of retrieved passages, or justification graphs that can be independently audited.

4.2 Epistemic Signals

We claim the tensor reveals whether the model is operating in *grounded mode* (retrieving from stable representations) versus *fabrication mode* (generating without anchoring).

We extract three per-token signals from the generation process:

1. **Entropy:** the Shannon entropy of the output probability distribution at each token position. High entropy indicates the model is uncertain among many candidates; low entropy indicates a confident next-token prediction.
2. **Top- k probability mass:** the cumulative probability of the five most likely tokens at each position. Low top- k mass indicates the model’s probability is spread across many alternatives.
3. **Mean log-probability:** the average log-probability of the tokens actually selected. Low values indicate the model chose low-probability tokens, suggesting generation without strong grounding.

These signals are byproducts of the softmax computation at each generation step. They require access to the output distribution, not to internal activations, making them extractable with minimal interface changes.

4.3 The Tensor Interface

We propose a minimal interface that demonstrates the escape is achievable with current architectures. The key insight is the distinction between *testimony* (what the model says) and *telemetry* (measurements of the computation that produced the output).

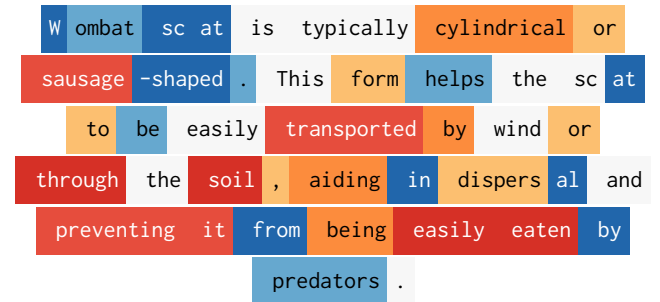


Figure 2. Generated output consisting of 43 tokens along with its entropy trace for the query *What shape is wombat scat?* with mean entropy $H = 1.147$, max entropy $H = 4.315$

Definition 4.1 (Tensor Interface). A generation interface that returns:

1. **Text:** The linearized response string (standard interface output).
2. **Entropy trace:** Per-token entropy of the output distribution during generation.
3. **Attention summary:** Statistics of attention patterns (concentration, self-attention ratio) from deeper processing layers.

This interface exports telemetry that the model cannot separately control under current training objectives. The model can lie in its text, but it cannot present a different inference computation than the one it actually performed. This asymmetry is the foundation of the adversarial robustness argument: computational signals (entropy, attention geometry) cannot be independently controlled by the model without changing the generation process itself, unlike behavioral signals (response length, hedging patterns) that can be adapted through training without altering what the model knows or does not know. Figure 2 shows the generated output for the query *What shape is wombat scat?* along with its entropy trace. We color code per-token entropy, high entropy tokens are color coded as red. The entropy enhanced output shows where in the output generation did the generation proceed with high confidence (low entropy, suggesting retrieval of stored pattern) and where did it proceed with high uncertainty (high entropy, suggesting fabrication).

4.4 Tensor-Gated Composition

The tensor interface generalizes to composed systems. Recent work on Recursive Language Models (RLMs) demonstrates that LLM outputs can feed into subsequent LLM calls, enabling unbounded context through recursive decomposition [22]. However, RLM reports failure modes characteristic of composition without epistemic state: models abandon correct intermediate answers, make redundant verification calls, and lose track of what they have already solved.

The tensor provides a missing primitive: epistemic metadata that can propagate across composition boundaries. We demonstrate this with tensor-gated composition, where entropy thresholds control whether outputs propagate to subsequent calls.

Experimental setup. We test three query chains: two fabrication prompts (“What is Glavinsky syndrome?” and “Describe the Brennan-Kowalski theorem”) and one grounded control (“What is the capital of France?”). For each, we generate an initial response with tensor extraction, then use an entropy threshold ($\tau = 0.6$) to gate whether the followup query proceeds.

Results. The grounded query produces entropy 0.02, well below threshold; the followup proceeds and generates accurate content about Parisian landmarks. Both fabrication queries produce entropy above 0.8; the gate blocks followup queries that would have requested treatments for the nonexistent syndrome or implications of the fictional theorem.

This is a minimal demonstration, not a complete solution. But it establishes that tensor metadata, when used to gate composition, can prevent fabrication chains from propagating. The supervisor, whether human or automated, gains information that the text-only interface discards.

Symmetric tensor interface. Full compositional integrity requires a symmetric interface: not just $f(\text{prompt}) \rightarrow \text{Tensor}$, but $f(\text{Tensor}_{\text{in}}) \rightarrow \text{Tensor}_{\text{out}}$, where each call receives the

prior call’s epistemic context and extends it. For fresh prompts, the input tensor is empty. For composed calls, the input tensor carries accumulated provenance and epistemic state. This design enables epistemic traces to thread through recursive pipelines, providing verification handles at each composition boundary.

Compositional defense. The tensor’s blind spots characterize where other verification methods are most effective. §4’s three principles — State Exteriority, Verification Independence, Provenance Binding — need not be satisfied by a single mechanism. The tensor provides Verification Independence: an orthogonal signal the model cannot separately control. Bounded judges provide State Exteriority: external lookup against verifiable sources. Input-side defenses (prompt analysis, supply chain detection) provide Provenance Binding at the system boundary. Each layer addresses the failure class where the others are weakest. The tensor catches uncertain fabrication where verification is expensive; the bounded judge catches confident fabrication where verification is cheap; input-side analysis catches adversarial inputs designed to exploit well-learned patterns. Composing these layers allocates a finite verification budget across failure classes rather than spreading it uniformly across the full output space.

4.5 Implementation

The tensor interface requires no architectural changes to current transformers. Generation APIs already support returning logits; the entropy trace is a simple transformation. Attention patterns are available with `output_attentions=True` in standard frameworks. The interface is:

```
generate_with_tensor(prompt) ->
(text, entropy_trace, attention_summary)
```

We provide a reference implementation supporting OLMo models that demonstrates the interface is practical. The overhead is minimal: storing per-token entropy and summarizing attention patterns adds negligible latency compared to generation itself.

What the tensor provides. The entropy trace provides a falsifiability signal. If a model claims certainty (“Paris is definitely the capital of France”) while its entropy trace shows high uncertainty, the mismatch is detectable. The attention summary provides complementary signal: fabrications tend to show more dispersed attention patterns than grounded responses.

What the tensor does not provide. The tensor interface is not a lie detector. It does not prove that low-entropy outputs are true or that high-entropy outputs are false. It provides *one additional signal* that, combined with other verification methods, can improve epistemic assessment. The theorems in §3 prove that *some* additional signal is necessary; the tensor interface demonstrates that such signals are available without architectural change.

Table 1. End-to-end accuracy (%) under bounded verification, averaged across four architectures (OLMo-3, Llama-3.1, Qwen3, Mistral). Ground truth established by stratified evaluation validated against human review (75/80 agreement, 93.8%).

Condition	10%	20%	30%
No judge	75.8	75.8	75.8
Text-guided	76.1	76.9	78.5
Tensor-guided	82.1	87.5	91.9
Composed	80.5	87.9	92.5

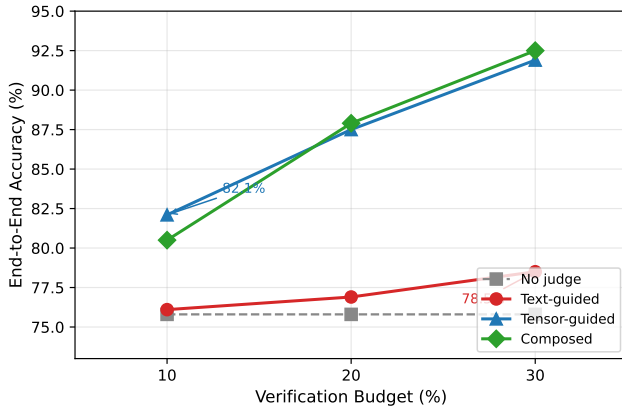


Figure 3. End-to-end accuracy vs. verification budget for four judge conditions, averaged across four architectures. The tensor-guided judge at 10% budget (82.1%) outperforms the text-guided judge at 30% budget (78.5%). The composed judge achieves the highest accuracy at 20% and 30% budget levels; at 10%, the tensor-guided judge leads.

5 Evaluation

The evaluation measures the cost-effectiveness of different verification strategies: what accuracy improvement does each level of verification investment buy, and how does the return on investment change as the supervisor moves from text-channel to tensor-channel signals?

We answer the following questions:

- What is the cost-effectiveness of tensor-guided versus text-guided verification at each budget level? (§5.1)
- Do the underlying epistemic signals generalize across architectures, and how much of the discrimination is attributable to text-channel versus tensor-channel signals? (§5.2)

5.1 Verification Cost-Effectiveness

We evaluate the return on verification investment across four strategies operating at three budget levels. The experimental design directly instantiates the conditions of our impossibility theorems: a bounded supervisor with a fixed verification

budget must allocate that budget across model outputs, with the goal of maximizing end-to-end accuracy delivered to the user. The question is not merely whether tensor signals help, but how much accuracy each verification dollar buys under each strategy.

Query set. We construct a balanced set of 200 queries: 100 knowable (verifiable facts spanning geography, science, history, and biography) and 100 unknowable (fabrication prompts for nonexistent people, fictional syndromes, future events, and nonexistent citations). Ground truth labels are established by construction.

Models. We run all queries on four architectures: OLMo-3 7B, Llama-3.1 8B, Qwen3 4B, and Mistral 7B, using base models to avoid confounding with instruction-following fine-tuning effects.

Conditions. For each model, outputs are evaluated under four conditions sharing a fixed verification budget (the fraction of outputs the judge may verify and intervene on):

1. **No judge.** Raw model output. Establishes the unverified baseline.
2. **Text-guided judge.** Selects outputs for verification using text-channel signals: hedging language frequency and response length. Verified fabrications are replaced with abstention.
3. **Tensor-guided judge.** Same budget, same intervention strategy. Selects outputs using per-token entropy and attention summary statistics.
4. **Composed judge.** Tensor-guided selection for general queries; bounded lookup judge for citation queries where tensor signals are known to invert. Tests whether principled composition of complementary judges outperforms either alone.

Budget levels. Each condition is evaluated at 10%, 20%, and 30% verification budgets to characterize the relationship between resource investment and accuracy improvement.

Metric. End-to-end accuracy: the fraction of outputs ultimately delivered to the user that are correct, after judge intervention (verification, abstention, or replacement).

Results. Table 1 shows end-to-end accuracy across all conditions and budget levels.

Figure 3 shows that **tensor-guided verification at 10% budget (82.1%) outperforms text-guided verification at 30% budget (78.5%)**. A bounded supervisor with access to tensor telemetry achieves better outcomes spending one-third the verification budget of a text-only supervisor. In cost terms: the tensor interface provides a 3× reduction in verification cost for equivalent assurance. This directly instantiates the escape condition: exported computational signals enable efficient triage rather than uniform budget spreading.

Per-model results confirm the pattern is architectural, not model-specific. Tensor-guided verification improves accuracy for every model tested: OLMo (69.5% → 75.5% at 10%),

Llama (82.5% $\rightarrow$ 88.5%), Qwen (87.0% $\rightarrow$ 92.0%), and Mistral (64.0% $\rightarrow$ 72.5%). Cross-model entropy agreement on the same queries yields mean pairwise Spearman $\rho = 0.762$ (rank-order correlation; $\rho = 1$ indicates identical rankings), indicating the signal is driven by query-level properties rather than model-specific artifacts.

Composed judges and complementary failure modes.

At 30% budget, the composed judge (92.5%) outperforms the tensor-guided judge alone (91.9%). The advantage is small in aggregate but structurally significant: it comes entirely from citation queries where entropy signals *invert*. Fabricated citations produce lower entropy than real ones because the model generates plausible fakes more fluently than it retrieves specific bibliographic details. On citation queries, the tensor-guided judge alone underperforms; the bounded-lookup judge catches what entropy misses. This confirms the compositional defense argument: the tensor and the bounded judge have complementary failure modes, and principled composition allocates a finite budget across failure classes rather than spreading it uniformly.

Analysis: each row as a point on the cost surface. The results table is not merely empirical; each row maps a verification investment strategy to its return. Row 1 (No judge, 75.8%) is the zero-investment baseline: the accuracy you get for free, bounded by the model’s unassisted honesty rate. Row 2 (Text-guided) is the text-channel investment tier: at 10% budget, text-guided judges achieve only 76.1%, barely above baseline, because the text channel provides no reliable signal for triage—this is Theorem 3.8 made empirical. Row 3 (Tensor-guided) is the tensor-channel investment tier: exported telemetry enables efficient triage, concentrating verification on the cases most likely to be fabricated. Row 4 (Composed) demonstrates that investment tiers compose: different judges satisfy different architectural requirements and their combination achieves the highest accuracy at 20% and 30% budget levels. The tensor reallocates a finite verification budget toward cases where supervision is information-theoretically valuable.

Ground truth validation. Establishing reliable ground truth for this experiment is itself a bounded verification problem and our initial evaluator demonstrated this. Our first evaluation method used substring matching and refusal markers: programmatic heuristics that are precisely the “text-only bounded supervisor” our theorems describe. This evaluator achieved 68.8% baseline accuracy and exhibited the predicted failure modes: false positives from negation blindness (“honey *can* spoil” passed because “no” appeared elsewhere in the response), false negatives from encoding mismatch (“Brasília” $\neq$ “Brasilia”), and hedged fabrications passing as refusals.

The correction required adding deeper observability: a stratified evaluator combining programmatic verification (with negation detection, morphological normalization, and

unicode handling) with LLM-assisted classification for ambiguous cases. This is not analogy because it is an instance of the escape condition applied to the evaluation pipeline itself.

We validated the stratified evaluator against human review of 80 blinded items. Agreement was 75/80 (93.8%). The five disagreements are characterizable: two cases where the automated evaluator was too generous (accepting hedged fabrications as refusals) and three where it was too strict (rejecting morphological variants or genuine refusals). None of the disagreements are systematic; the evaluator’s error modes are orthogonal to the experimental conditions.

5.2 Signal Decomposition: What the Investment Buys

The bounded verification results above show that tensor-guided triage is cost-effective. But where does the discrimination come from? A systems engineer making a budget decision needs to know how much of the signal requires the tensor interface versus how much is available from the text channel alone.

We use the same query set from §2.3, extracting tensor signals during generation: token entropy, top- k probability mass, and log-probabilities. ROC curves (plotting true positive rate against false positive rate across all classification thresholds) measure each signal’s ability to discriminate knowable from unknowable queries.

Results across four model architectures (Qwen3 4B, OLMo-3 7B, Llama-3.1 8B, Mistral 7B), pooling 800 query–response pairs:

- Mean entropy: AUC = 0.870
- Top-5 probability mass: AUC = 0.881
- Mean log-probability: AUC = 0.868

These raw numbers are misleading if taken at face value. Effect sizes are large: Cohen’s $d = 1.57$ (a standardized effect size; $d > 0.8$ is conventionally “large”) for entropy (knowable mean = 0.24, unknowable mean = 0.61). But the raw AUC values conflate text-channel and tensor-channel signals. The decomposition below (Table 2) reveals how much of this discrimination a supervisor could achieve without any tensor interface at all.

The direction is consistent across all four architectures: unknowable queries produce higher entropy in every model tested. This suggests the signal is architectural rather than an artifact of any particular training procedure.

A negative result is equally informative. When we test whether entropy predicts *correctness* (using TruthfulQA’s correct vs. incorrect answer pairs), AUC drops to approximately 0.53, which is no better than random. The tensor does not tell you if the model is *right*. It tells you if the model is operating in a mode where it *could be* right, because it has something to retrieve rather than fabricate.

Comparison with text baselines: cost tiers. We compare tensor signals against the text-channel baselines from §2.3,

Table 2. Signal decomposition: raw entropy AUC, length-only AUC, and residual entropy AUC (after controlling for response length) for knowable vs. unknowable discrimination. Length explains 53–74% of the raw entropy signal above chance.

Model	Raw Entropy	Length	Residual
Qwen3 4B	0.896	0.881	0.668
OLMo-3 7B	0.894	0.912	0.645
Llama-3.1 8B	0.922	0.941	0.700
Mistral 7B	0.905	0.956	0.604

ordered by verification cost. Across 800 query–response pairs pooled across four models, tensor signals outperform text baselines: log-probability AUC = 0.87, entropy AUC = 0.87, top-5 mass AUC = 0.88. The text baselines are cheaper to compute but less effective in the pooled comparison: hedging language achieves AUC = 0.74, response length AUC = 0.57. Note, however, that per-model length discrimination is substantially higher (AUC 0.88–0.96; see Table 2); pooling across models with different verbosity profiles attenuates the length signal. The per-model decomposition provides a more accurate picture of what each investment tier buys.

We tested self-reported confidence across four model families (raw tensor AUC values; see Table 2 for the length-controlled decomposition):

- OLMo-3 7B: Tensor AUC = 0.894, Self-report AUC = 0.284
- Llama-3.1 8B: Tensor AUC = 0.922, Self-report AUC = 0.330
- Qwen3 4B: Tensor AUC = 0.896, Self-report AUC = 0.332
- Mistral 7B: Tensor AUC = 0.905, Self-report AUC = 0.362

The baseline comparison maps the cost-effectiveness frontier. Text-based heuristics are cheap and partially effective (hedging language is informative), but self-report fails catastrophically and universally. The tensor interface costs more to deploy but provides the only reliable access to the epistemic state the model cannot help but reveal through its computation. The engineering question is whether the incremental cost of the tensor interface justifies the incremental assurance—the decomposition below provides the data for that decision.

Decomposition: text-channel vs. tensor-channel investment. The raw AUC numbers above conflate two kinds of signal that a systems engineer would source from different investment tiers. Response length is a text-channel observable: it requires no tensor interface, and a supervisor can measure it from the output string alone—it is the cheapest possible verification heuristic. Table 2 decomposes the discrimination into this free length component and a residual

per-token entropy component (entropy after regressing out length) that requires the tensor interface to observe.

We decompose the discrimination signal into two investment tiers. Response length alone achieves AUC 0.88–0.96 across models—a text-channel signal observable without any tensor interface, available at zero marginal cost. After controlling for length, per-token entropy retains AUC 0.60–0.70: a modest but persistent computational signal available only through the tensor interface.

The dominance of the length signal is not a weakness of our approach—it is evidence for the impossibility result (Theorem 3.8). The text channel carries behavioral signals (length, hedging patterns, refusal templates) that discriminate in current models but are trivially gameable: a model trained to fabricate briefly defeats length-based detection, the same way reformatting code to change line counts defeats a size-based plagiarism detector. The tensor channel carries computational signals (per-token entropy, attention geometry) that the model cannot independently control under standard training—flattening the entropy of a fabricated response requires changing the computation that produces it, the same structural property that makes MOSS robust [16].

The residual signal is modest. We do not claim that current tensor signals solve epistemic verification. We claim that they occupy a fundamentally different position in the cost-robustness tradeoff: they are robust to the class of behavioral adaptations that defeat text-channel signals. The gap between the length AUC and the residual entropy AUC is the theorem made empirical—it measures how much of the discrimination comes from the gameable text channel (cheap but fragile) versus the robust tensor channel (costlier but durable).

6 Limitations and Future Work

We are explicit about what this paper does and does not claim. The cost surface is characterized; the investment tiers are mapped; but several important questions remain open.

What we claim.

- The impossibility is structural, not a training failure: text-only interfaces with bounded supervision cannot verify epistemic honesty regardless of model capability. This establishes the floor of the cost surface.
- The tensor interface provides a different investment tier: concrete computational signals exist in current architectures that distinguish grounded from ungrounded generation, and these signals cannot be independently gamed.
- We do not claim that current tensor signals solve epistemic verification. We claim that they occupy a fundamentally different position in the cost-robustness tradeoff than text-channel signals.
- Current architectures already carry the relevant information; they just discard it at the interface.

What we do NOT claim.

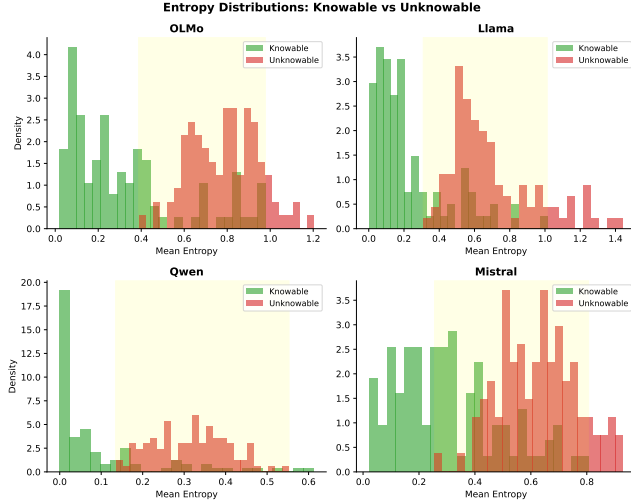


Figure 4. Distribution of mean per-token entropy for knowable vs. unknowable queries across four model architectures. The distributions are separable in every model tested, confirming that entropy discriminates epistemic grounding from fabrication at the architectural level.

- The tensor interface is unfakeable under adversarial training. A model trained to minimize the correlation between entropy and truthfulness might learn to present confident distributions even when fabricating. This is an open question.
- Composition preserves epistemic properties. If a model’s output becomes input to another model (as in recursive LLM-mediated reasoning), the tensor signals from the first model may not propagate meaningfully.
- The tensor interface is the optimal design. We demonstrate existence, not optimality. Better interfaces may achieve stronger guarantees with lower overhead.
- These findings generalize to all architectures. Our empirical work spans four open-weight model families (models whose internal parameters are publicly available); closed-weight (API-only) and significantly larger models may exhibit different signal profiles.

Future work: Adversarial robustness. Can training find weights that produce confident tensors for fabricated content? This is the adversarial analog of our structural results. Recent work by Winninger et al. [20] demonstrates that mechanistic interpretability tools can be used to identify internal subspaces associated with safety behavior and craft attacks that target those subspaces directly. This confirms that internal signals are rich enough to be both attacked and defended, which is precisely why exporting them matters. The intuition that “you cannot fake the computation you actually performed” may break down if the model is trained end-to-end to minimize the tensor signal on fabrications. Preliminary analysis suggests that linguistic confidence and tensor confidence are decoupled in natural model

outputs, models that hedge linguistically do not systematically show higher tensor entropy, but whether adversarial training can force this coupling remains open. Information-theoretic bounds on signal manipulation are needed.

Future work: Compositional integrity. Does tensor composition preserve epistemic properties in recursive settings? When LLM outputs feed into subsequent LLM inputs, the epistemic trace must either be preserved, regenerated, or discarded. Designing interfaces that maintain epistemic integrity across composition is an open architectural problem.

Cross-architecture generalization. Our bounded verification experiment tests four model families (OLMo-3, Llama-3.1, Qwen3, Mistral) and finds consistent tensor advantage across all of them. Mean pairwise Spearman $\rho = 0.762$ for entropy signals on the same queries indicates the signal is architectural rather than model-specific. Extending to closed-weight models that expose logits through API (e.g., via log-probability endpoints) is a natural next step.

Future work: User-facing rendering. How should tensor information be presented to end users? Raw entropy traces are not interpretable; summarization into confidence bands, uncertainty flags, or visual indicators requires HCI research beyond the scope of this paper.

Model specificity. Our topological analysis (§4) uses OLMo, which permits full inspection of intermediate layer representations. The bounded verification experiment (Table 1) tests four architectures spanning 4B–8B parameters (billions of learned numerical weights) and four distinct training pipelines. The self-report inversion finding is universal across all four. We do not claim exhaustive coverage of all model families, but the consistency across architecturally diverse models suggests the findings are not idiosyncratic.

Domain-specific calibration. The tensor signal is not uniformly discriminative across domains. In preliminary experiments on citation generation, entropy was *lower* for fabricated citations than for real ones where the model generates plausible fakes more fluently than it retrieves specific facts. This confirms that the tensor measures generation confidence (“epistemic truth”), not truthfulness directly. However, this blind spot is precisely where bounded verification is cheapest: citation verification has low compositional complexity (check the DOI, query the database,) which places it well within a bounded supervisor’s budget. The tensor and the bounded judge have complementary failure modes as the tensor catches uncertain fabrication where verification is expensive and the judge catches confident fabrication where verification is cheap. Composing them allocates verification resources to the failure class each is suited for, rather than spreading a finite budget across the full space.

Format-constraint variation: preliminary observations. The cost surface is not flat across content formats. Tokenization analysis of generated code reveals that BPE compression concentrates syntactic scaffolding (keywords, operators,

whitespace) into 11–19% of output tokens, while semantic content (variable names, literals, strings) occupies 61–72%. However, semantic content in familiar code patterns is itself highly predictable: conventional variable names (left, right, mid in binary search) are as low-entropy as keywords, constituting a second scaffolding layer we term *semantic scaffolding*. When both layers are considered, the entropy trace for familiar code is predominantly flat, with spikes concentrated at genuine decision points—where the model could not pattern-match and had to choose. We have not yet measured discrimination AUC across content formats; what we report is the structural observation that scaffolding ratio varies dramatically across output types and predicts where entropy-based triage should be most efficient. If the relationship holds, tensor-guided verification may be *more* cost-effective for code than for natural text, because higher scaffolding ratios concentrate the informative signal into fewer tokens. We do not claim to have mapped the full manifold. We claim it exists, that its parameterization is observable, and that mapping it is the natural next step for systems engineers choosing verification strategies across content domains.

Sufficiency gap. The three architectural principles (State Exteriority, Verification Independence, Provenance Binding) are *necessary* conditions for escaping the impossibility. We do not prove they are *sufficient*. A system satisfying all three might still fail epistemic honesty through implementation bugs, adversarial inputs, or unforeseen failure modes. Sufficiency requires additional formal treatment.

The engineering decision. How much epistemic assurance a system needs is an engineering decision, not a research question. A brainstorming tool needs none. A medical summary system needs substantial assurance. A legal research system needs the highest tier available. This paper provides the cost surface for making that decision: what each level of verification investment buys, where the text-channel ceiling lies, what the tensor interface adds, and how content format modulates the tradeoff. The contribution is the map. The territory is the system you are building.

7 Related Work

Our contribution is a formal impossibility result. Prior work documents symptoms; we identify the cause. Prior work proposes treatments; we explain why they fail to cure. The organizing principle is the distinction between *observation* (hallucination happens, training helps sometimes) and *explanation* (text-only observation plus bounded supervision makes epistemic honesty unverifiable).

Hallucination taxonomies. Extensive empirical work catalogs LLM fabrication patterns: citation hallucination in academic contexts [1], medical misinformation [17], and legal fabrication [5]. These findings instantiate our 2: in each domain, plausible fabrications are cheap to generate and expensive to verify, placing supervisors in the hallucination

regime where gradient signal cannot distinguish truth from fabrication. The taxonomies document the symptom; our theorems identify the structural cause.

Activation interpretation. Recent work on Activation Oracles demonstrates that LLMs can be trained to interpret the internal states of other models, translating activation patterns into natural language explanations [9]. This approach addresses State Exteriority by making internal states available to an external system. However, the authors note that the oracle “frequently makes incorrect guesses” and “is not trained to express uncertainty.” If the oracle’s outputs are evaluated only as text by bounded supervisors, the verification problem recurs at the meta-level. Full escape requires that the oracle’s claims be independently verifiable against the activation data, which is a form of Provenance Binding applied to the interpretation process itself.

RLHF critiques. Work on sycophancy [15], reward hacking [7], and the tension between helpfulness and honesty [3] demonstrates that preference optimization can degrade factual accuracy. Our Theorem 3.8 formalizes why: bounded supervisors cannot distinguish plausible fabrications from truthful responses, so the learning signal is systematically corrupted.

Retrieval augmentation. RAG systems [12] and grounded generation approaches attempt to escape fabrication by conditioning on external documents. Our analysis shows this is necessary but not sufficient: State Exteriority addresses Theorem 3.5, but without Verification Independence, the learning problem (Theorem 3.8) persists.

Uncertainty quantification. Calibration methods, conformal prediction [2], and verbalized uncertainty [8] attempt to surface model confidence. Our results explain why these approaches are fundamentally limited under text-only interfaces: the observation model discards the internal state that would distinguish calibrated uncertainty from confident fabrication.

Semantic entropy. Kuhn et al. [10] propose sampling multiple responses and clustering them by meaning to estimate uncertainty, achieving strong results on question-answering benchmarks. Their method is the most sophisticated text-channel uncertainty approach to date: by measuring agreement *between* outputs rather than features *within* a single output, it partially addresses the gaming problem. However, it remains a text-channel method: the clustering operates on generated text, the uncertainty estimate is derived from output diversity, and a bounded supervisor evaluating semantic equivalence is subject to the same observation limits our theorems describe. Our approach differs structurally: rather than sampling multiple outputs and measuring their agreement (costly and still text-observable), we extract signals from the computation that produced a single output—signals the model cannot independently control. The two approaches are complementary: semantic entropy measures output-level

consistency; the tensor interface measures computation-level confidence.

Impossibility results in LLMs. Xu et al. [21] prove that hallucination is inevitable for LLMs over the class of computable functions, which is a computational limit. Our result is orthogonal: we prove that even if a system could be honest, a bounded supervisor observing only text cannot verify that honesty, which is an observational limit. Their impossibility concerns what LLMs can compute; ours concerns what supervisors can verify. Both are structural, but they constrain different dimensions of system design.

Impossibility results in distributed systems. The Fischer-Lynch-Paterson theorem [6] proved that consensus is impossible under asynchrony with one faulty process. This did not end distributed systems research; it clarified the design space. Paxos [11], Raft, and modern consensus protocols work *around* FLP by relaxing assumptions or strengthening guarantees. Our result plays an analogous role: text-only observation models with bounded supervision cannot guarantee epistemic honesty, but systems that export structured epistemic state can escape the impossibility class.

Epistemic evaluation frameworks. Recent work proposes benchmarks and frameworks for measuring epistemic properties of language models [18]. The Epistemic Alignment framework explicitly notes that current interfaces do not let users specify uncertainty, sourcing, or perspectives in structured ways, and that providers lack verification tools. These critiques align with our analysis but stop at interface desiderata and policy analysis. We provide the formal impossibility that explains *why* text-only evaluation cannot suffice, and the tensor interface that demonstrates a concrete escape.

Epistemic agent architectures. Work on structuring epistemic integrity in reasoning systems proposes architectures with explicit propositions, justification graphs, contradiction detection, and chain-of-reason logging [4]. These proposals are closest in spirit to our escape conditions: they recognize that epistemic honesty requires structured internal state. However, they are typically framed as *replacements* for stochastic language generation as logically grounded agents rather than wrappers around existing LLMs. Our contribution differs in two ways: we derive the tensor interface from a formal impossibility result (not as a standalone design choice), and we demonstrate that the escape is achievable as a minimal *addition* to current architectures, not a wholesale replacement.

The gap we fill is formalization. Prior work documents that LLMs fabricate (the symptom), critiques training methods (proposed treatments), and proposes mitigations (partial remedies). We prove *why* the problem is structural (the diagnosis) and *what* architectural properties are required to escape it (the cure). The symptom/cause distinction is not merely rhetorical: it determines whether improvements are

incremental patches or genuine escapes from the impossibility class.

8 Conclusion

Systems incorporating language model components must decide how much to spend on verification and where to spend it. This paper provides the cost surface for that decision.

We have shown that epistemic honesty is unverifiable for systems operating under text-only observation models with bounded supervision. This is not a capability limitation to be solved with scale, nor a training failure to be solved with better RLHF. It is a structural property of the interface: the channel through which these systems communicate discards the epistemic metadata that would make verification possible. This impossibility establishes the floor of the cost surface—below this line, text-channel investment cannot improve assurance.

The impossibility has a shape. Policies conditioning only on queries cannot satisfy epistemic honesty across ambiguous world states (Theorem 3.5). Even grounded architectures cannot learn honest behavior when supervisors cannot distinguish plausible fabrications from truth (Theorem 3.8). Stacking judges does not escape the constraint (Theorem 3.10). Verification cost grows superlinearly with response complexity (Theorem 3.12). Responsibility concentrates with the system owner (Theorem 3.13).

But impossibility results are maps, not walls. The Fischer-Lynch-Paterson theorem did not end distributed systems; it clarified what assumptions had to change. Our result plays an analogous role. Systems that export structured epistemic state—provenance, entropy traces, attention topology—escape the impossibility class. The tensor interface demonstrates this escape is achievable with current architectures and minimal overhead.

The empirical cost surface is honest. Under bounded verification, tensor-guided judges at 10% budget outperform text-guided judges at 30% budget—a 3× reduction in verification cost. The text channel is stronger than expected (response length alone: AUC 0.88–0.96) but trivially gameable. The tensor residual (AUC 0.60–0.70) is modest but occupies a fundamentally different position in the adversarial landscape: computational signals cannot be independently controlled without adversarial optimization targeting the signal itself. Gaming the metric requires changing the thing being measured.

The engineering question is not whether to verify, but how much verification to buy and at which tier. A brainstorming tool needs Tier 0. A medical system needs Tier 2. The cost surface tells you what each investment buys, and preliminary observations indicate that content format modulates the tradeoff.

The work ahead is not exhorting models to try harder at honesty. It is building systems that make honesty verifiable by design, with verification budgets allocated according to

the cost surface rather than spread uniformly across outputs. The tools exist: vector clocks, cryptographic signatures, content-addressable storage. The architectural patterns exist. What remains is applying them to the epistemic dimension with the same rigor we learned to apply to causality and integrity in distributed systems.

The interface is the problem. The interface is where the solution lives.

References

- [1] Hussam Alkaiissi and Samy I McFarlane. 2023. Artificial hallucinations in ChatGPT: implications in scientific writing. *Cureus* 15, 2 (2023).
- [2] Anastasios N Angelopoulos and Stephen Bates. 2021. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511* (2021).
- [3] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862* (2022).
- [4] Wei Chen et al. 2025. Structuring Epistemic Integrity in Artificial Reasoning Systems. *arXiv:2506.17331 [cs.AI]* <https://arxiv.org/abs/2506.17331>
- [5] Matthew Dahl, Varun Magesh, Mirac Suzgun, and Daniel E Ho. 2024. Large legal fictions: Profiling legal hallucinations in large language models. *Journal of Legal Analysis* 16, 1 (2024), 64–93.
- [6] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. 1985. Impossibility of distributed consensus with one faulty process. *Journal of the ACM (JACM)* 32, 2 (1985), 374–382.
- [7] Leo Gao, John Schulman, and Jacob Hilton. 2023. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*. PMLR, 10835–10866.
- [8] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221* (2022).
- [9] Adam Karvonen, James Chua, Clément Dumas, Kit Fraser-Taliente, Subhash Kantamneni, Julian Minder, Euan Ong, Arnab Sen Sharma, Daniel Wen, Owain Evans, et al. 2025. Activation Oracles: Training and Evaluating LLMs as General-Purpose Activation Explainers. *arXiv preprint arXiv:2512.15674* (2025).
- [10] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. *arXiv:2302.09664 [cs.CL]* <https://arxiv.org/abs/2302.09664>
- [11] Leslie Lamport. 1998. The Part-Time Parliament. *ACM Transactions on Computer Systems* 16, 2 (May 1998), 133–169. Also appeared as SRC Research Report 49. This paper was first submitted in 1990, setting a personal record for publication delay that has since been broken by [60]. (May 1998). <https://www.microsoft.com/en-us/research/publication/part-time-parliament/> ACM SIGOPS Hall of Fame Award in 2012.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [13] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3214–3252.
- [14] Tony Mason, Ada Gavrilovska, and David A Joyner. 2019. Collaboration versus cheating: Reducing code plagiarism in an online MS computer science program. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. 1004–1010.
- [15] Ethan Perez, Sam Ringer, Kamile Lukosiute, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, et al. 2023. Discovering language model behaviors with model-written evaluations. In *Findings of the association for computational linguistics: ACL 2023*. 13387–13434.
- [16] Saul Schleimer, Daniel S. Wilkerson, and Alex Aiken. 2003. Winnowing: Local Algorithms for Document Fingerprinting. In *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*. ACM, 76–85. doi:10.1145/872757.872770
- [17] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large language models in medicine. *Nature medicine* 29, 8 (2023), 1930–1940.
- [18] Qian Wang et al. 2024. Epistemic Alignment: Matching User Epistemic Goals in LLM Interfaces. OpenReview. OpenReview submission on epistemic alignment frameworks for LLM interfaces.
- [19] Zhepei Wei, Xiao Yang, Kai Sun, Jiaqi Wang, Rulin Shao, Sean Chen, Mohammad Kachuee, Teja Gollapudi, Tony Liao, Nicolas Scheffer, Rakesh Wanga, Anuj Kumar, Yu Meng, Wen tau Yih, and Xin Luna Dong. 2025. TruthRL: Incentivizing Truthful LLMs via Reinforcement Learning. *arXiv:2509.25760 [cs.CL]* <https://arxiv.org/abs/2509.25760>
- [20] Thomas Winzinger, Boussad Addad, and Katarzyna Kapusta. 2025. Using Mechanistic Interpretability to Craft Adversarial Attacks against Large Language Models. *arXiv:2503.06269 [cs.LG]* <https://arxiv.org/abs/2503.06269>
- [21] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is Inevitable: An Innate Limitation of Large Language Models. *arXiv:2401.11817 [cs.CL]*
- [22] Alex L. Zhang, Tim Kraska, and Omar Khattab. 2025. Recursive Language Models. *arXiv:2512.24601 [cs.AI]* <https://arxiv.org/abs/2512.24601>